



Test: Seguridad Básica en Aplicaciones Web

Puntuación: 50/50

Correctas: 50

Incorrectas: 0

Pregunta 1

¿Cuál es la regla de oro para el control de permisos en aplicaciones web?

- ☐ A. Validar permisos solo en el frontend
- ☒ B. Validar permisos en el backend siempre
- ☐ C. Los permisos se validan en la base de datos únicamente
- ☐ D. Ocultar botones es suficiente para la seguridad

 Retroalimentación:

El control de permisos SIEMPRE se debe hacer en el backend. El frontend puede ocultar elementos, pero quien manda es el servidor. Un usuario malicioso puede manipular el frontend fácilmente.

Pregunta 2

En una ruta como /pedidos/2, ¿qué debería verificar el backend antes de mostrar los datos?

- ☐ A. Solo que el pedido existe
- ☐ B. Solo la sintaxis de la URL
- ☐ C. Únicamente que el ID es válido
- ☒ D. Que el pedido pertenece al usuario autenticado

 Retroalimentación:

Es fundamental verificar que el pedido pertenece al usuario logueado. Esto previene que usuarios accedan a datos de otros usuarios simplemente cambiando el ID en la URL.

Pregunta 3

¿Por qué nunca debes guardar contraseñas en texto plano en la base de datos?

- ☐ A. Las bases de datos no las aceptan
- ☒ B. Son vulnerables si alguien accede a la BD
- ☐ C. No se pueden comparar después
- ☐ D. Ocupan más espacio

 Retroalimentación:

Si alguien accede a tu base de datos y las contraseñas están en texto plano, puede ver todas las contraseñas de los usuarios. Usar hash hace que sean ilegibles incluso si se compromete la BD.

Pregunta 4

¿Qué librería se recomienda para hashear contraseñas en Node.js?

- ☐ A. md5
- ☒ B. bcryptjs

- C. sha1
- D. crypto

 Retroalimentación:

bcryptjs es la librería recomendada porque está específicamente diseñada para hashear contraseñas de forma segura. MD5 y SHA1 son vulnerables para contraseñas.

Pregunta 5

¿Cuál es el método correcto para verificar una contraseña con bcrypt?

- A. bcrypt.hash(password, hash)
- B. bcrypt.compare(password, hash)
- C. bcrypt.check(password, hash)
- D. bcrypt.verify(password, hash)

 Retroalimentación:

bcrypt.compare() es el método que compara una contraseña en texto plano con el hash almacenado, devolviendo true si coinciden y false si no.

Pregunta 6

¿Cuántos tipos de mensajes principales hay en una aplicación?

- A. Mensajes UX y logs para desarrolladores
- B. Solo logs técnicos
- C. Tres tipos diferentes
- D. Solo mensajes de error

 Retroalimentación:

Hay dos tipos principales: mensajes UX para el usuario final (claros y orientados a acción) y logs para desarrolladores (con detalle técnico para depuración).

Pregunta 7

¿Dónde se ven los console.log() del frontend?

- A. En archivos de log del sistema
- B. En DevTools del navegador
- C. En la terminal del servidor
- D. No se ven en ningún lado

 Retroalimentación:

Los console.log() del frontend se ven en las DevTools del navegador, en la pestaña Console. Es útil para depuración durante el desarrollo.

Pregunta 8

¿Dónde se ven los console.log() del backend?

- A. En el navegador
- B. En archivos automáticos
- C. En la terminal donde corre Node.js
- D. En la interfaz de usuario

 Retroalimentación:

Los `console.log()` del backend aparecen en la terminal/consola donde está ejecutándose el proceso de Node.js (tu PC, servidor, logs del hosting).

Pregunta 9

¿Qué información NUNCA debes mostrar al usuario en producción?

- A. Mensajes de confirmación
- B. Mensajes de validación
- C. Notificaciones de éxito
- D. Stack traces completos y consultas SQL**

 Retroalimentación:

Nunca muestres stack traces completos, consultas SQL o rutas del servidor al usuario. Esto expone información sensible que podría ser utilizada por atacantes.

Pregunta 10

¿Qué tipo de información debe ir en el archivo `.env`?

- A. Configuración pública
- B. Contraseñas de BD y claves de API**
- C. Nombres de archivos
- D. URLs públicas

 Retroalimentación:

El `.env` debe contener información sensible como contraseñas de base de datos, `JWT_SECRET`, claves de API, etc. Nunca información pública.

Pregunta 11

¿El archivo `.env` se debe subir a Git?

- A. Depende del proyecto
- B. Solo en producción
- C. Sí, siempre
- D. No, nunca**

 Retroalimentación:

NUNCA subas `.env` a Git. Contiene información sensible que no debe ser pública. Úsalo en `.gitignore` para asegurarte de que Git lo ignore.

Pregunta 12

¿Qué es un script en el contexto de programación?

- A. Una base de datos
- B. Una página web
- C. Un lenguaje de programación
- D. Un archivo con instrucciones que se ejecutan automáticamente**

 Retroalimentación:

Un script es un archivo que contiene una serie de instrucciones que el ordenador ejecuta automáticamente, una tras otra, para automatizar tareas repetitivas.

Pregunta 13

¿Cómo se ejecuta un script de JavaScript con Node.js?

- A. `php script.js`
- B. Se ejecuta automáticamente
- C. `./script.js`
- D. `node script.js`

 Retroalimentación:

Los scripts de JavaScript se ejecutan con Node.js usando el comando 'node nombre_del_script.js' desde la terminal.

Pregunta 14

¿Cuál es la sintaxis básica de una consulta SELECT en SQL?

- A. `QUERY * FROM tabla`
- B. `SELECT * FROM tabla`
- C. `FETCH * FROM tabla`
- D. `GET * FROM tabla`

 Retroalimentación:

`SELECT * FROM tabla` es la sintaxis básica para consultar todos los campos de una tabla en SQL. `SELECT` especifica qué datos quieres, `FROM` indica de qué tabla.

Pregunta 15

¿Cuál es la función principal de los controladores en una aplicación Express?

- A. Validar solo los datos de entrada
- B. Renderizar las vistas únicamente
- C. Gestionar la lógica de negocio y coordinar entre rutas y modelos
- D. Conectar con la base de datos directamente

 Retroalimentación:

Los controladores separan la lógica de negocio de las rutas. Reciben datos del cliente, procesan la lógica (usando modelos si es necesario), y devuelven respuestas apropiadas, manteniendo el código organizado y reutilizable.

Pregunta 16

Cuando clonas un proyecto, ¿cuál es el primer comando que debes ejecutar?

- A. `npm start`
- B. `git pull`
- C. `npm install`
- D. `npm run build`

 Retroalimentación:

`npm install` es lo primero que debes hacer tras clonar. Instala todas las dependencias listadas en `package.json` y crea la carpeta `node_modules`.

Pregunta 17

¿Cuándo se debe generar la carpeta 'dist' en un proyecto React/Vite?

- A. Automáticamente con git
- B. Al instalar dependencias

- **C.** Cuando vas a desplegar (npm run build)

- D. Al clonar el proyecto

 Retroalimentación:

La carpeta 'dist' se genera con 'npm run build' cuando necesitas crear la versión optimizada para producción. No se incluye en Git porque es un archivo generado.

Pregunta 18

¿Qué significa que el frontend puede 'ocultar botones' pero 'quien manda es el servidor'?

- A. El frontend es más importante

- **B.** La seguridad real está en el backend, no en ocultar elementos

- C. Los botones controlan la seguridad

- D. El servidor maneja la interfaz

 Retroalimentación:

Ocultar un botón en el frontend no proporciona seguridad real. Un atacante puede hacer peticiones directas al servidor. La validación de permisos debe estar en el backend.

Pregunta 19

¿Para qué sirve la cláusula WHERE en una consulta SQL?

- A. Para unir tablas

- B. Para agrupar datos

- C. Para ordenar los resultados

- **D.** Para filtrar registros que cumplen una condición específica

 Retroalimentación:

WHERE se usa para filtrar registros en una consulta SQL. Solo devuelve las filas que cumplen la condición especificada, por ejemplo: `SELECT * FROM usuarios WHERE edad > 18.`

Pregunta 20

¿Cuál es el papel principal de los modelos en una arquitectura MVC?

- **A.** Gestionar los datos y la lógica de acceso a la base de datos

- B. Manejar las rutas de la aplicación

- C. Procesar únicamente las peticiones HTTP

- D. Renderizar la interfaz de usuario

 Retroalimentación:

Los modelos son responsables de la gestión de datos: definir estructura, validaciones, relaciones y operaciones CRUD con la base de datos. Encapsulan toda la lógica relacionada con los datos y proporcionan una interfaz limpia para que los controladores accedan a ellos.

Pregunta 21

¿Cuál es la diferencia entre 'git fetch' y 'git pull'?

- A. pull solo funciona con GitHub
- B. fetch descarga sin integrar, pull descarga e integra cambios
- C. Son exactamente lo mismo
- D. fetch es más lento que pull

 Retroalimentación:

git fetch descarga información del remoto sin tocar tu código local, mientras que git pull hace fetch + merge, integrando los cambios automáticamente en tu rama.

Pregunta 22

¿Para qué se usa la rama 'develop' en Git?

- A. Para almacenar archivos temporales
- B. Para hacer commits directamente
- C. Como zona de integración antes de pasar a main
- D. Solo para proyectos grandes

 Retroalimentación:

La rama develop sirve como zona de integración donde se combinan las distintas ramas feature antes de pasar los cambios estables a main/producción.

Pregunta 23

¿Qué es middleware en una aplicación Express?

- A. Solo para manejo de errores
- B. Un tipo de archivo
- C. Una base de datos
- D. Funciones que se ejecutan entre la petición y la respuesta

 Retroalimentación:

Middleware son funciones que se ejecutan en el flujo de petición-respuesta de Express. Pueden procesar datos, validar, autenticar, manejar errores o modificar req/res antes de llegar al controlador final.

Pregunta 24

¿Qué hace el comando 'git merge'?

- A. Sube cambios al remoto
- B. Elimina una rama
- C. Crea una nueva rama
- D. Integra cambios de una rama en otra

 Retroalimentación:

git merge integra los cambios de una rama en otra. Se ejecuta desde la rama que va a recibir los cambios (normalmente main o develop).

Pregunta 25

¿Cuándo aparecen conflictos de merge en Git?

- A. Cuando el repositorio es muy grande
- B. Cuando no tienes internet
- C. Cuando dos ramas modifican la misma línea del mismo archivo

- **D.** Solo cuando trabajas solo

 Retroalimentación:

Los conflictos de merge aparecen cuando dos ramas han modificado la misma línea del mismo archivo, y Git no puede decidir automáticamente qué versión conservar.

Pregunta 26

¿Qué contiene típicamente la carpeta 'dist'?

- **A.** Documentación del proyecto
- **B.** Solo archivos de configuración
- **C.** El código fuente del proyecto
- **D.** La versión compilada y optimizada para producción

 Retroalimentación:

La carpeta dist contiene la versión compilada, minificada y optimizada del frontend, lista para ser subida al servidor de producción.

Pregunta 27

¿Con qué comando se genera la carpeta 'dist' en un proyecto React/Vite?

- **A.** npm start
- **B.** npm run build
- **C.** npm install
- **D.** npm run dev

 Retroalimentación:

El comando 'npm run build' ejecuta el proceso de build que compila y optimiza el código, generando la carpeta dist con los archivos listos para producción.

Pregunta 28

¿Qué archivos NO deberían incluirse en el .gitignore?

- **A.** package.json y src/
- **B.** node_modules y .env
- **C.** Archivos temporales
- **D.** dist y logs

 Retroalimentación:

package.json y src/ contienen código fuente y configuración esenciales que deben versionarse. node_modules, .env, dist y logs se regeneran o contienen datos sensibles.

Pregunta 29

En trabajo colaborativo con Git, ¿por qué no se debe trabajar directamente en 'main'?

- **A.** Para mantener main estable y evitar romper el proyecto
- **B.** Porque es más lento
- **C.** No es cierto, siempre se debe trabajar en main
- **D.** Porque main solo es para administradores

Retroalimentación:

La rama main debe contener siempre código estable y funcional. Trabajar en ramas feature permite experimentar sin riesgo de romper la versión principal.

Pregunta 30

¿Cuál es el flujo profesional típico con ramas Git?

- A. develop → feature → main
- B. feature → develop → main
- C. Todas las ramas son iguales
- D. main → feature → develop

Retroalimentación:

El flujo profesional es: feature (desarrollo) → develop (integración) → main (producción). Cada etapa valida que el código funcione correctamente.

Pregunta 31

¿Qué hace el comando 'git clone'?

- A. Solo descarga archivos
- B. Sube cambios al servidor
- C. Crea una copia local completa del repositorio con su historial
- D. Actualiza un repositorio existente

Retroalimentación:

git clone crea una copia local completa del repositorio, incluyendo todo el historial de commits y la configuración para trabajar con el remoto.

Pregunta 32

¿Para qué sirve el archivo 'package.json' en un proyecto?

- A. Gestionar dependencias y scripts del proyecto
- B. Solo para decoración
- C. Almacenar contraseñas
- D. Configurar la base de datos

Retroalimentación:

package.json es fundamental: define las dependencias del proyecto, scripts de ejecución, metadatos y configuraciones necesarias para que npm funcione correctamente.

Pregunta 33

¿Cuál es la principal diferencia entre hosting compartido y VPS?

- A. En hosting compartido compartes recursos del servidor, en VPS tienes recursos dedicados
- B. Hosting compartido es más caro que VPS
- C. VPS es solo para sitios web estáticos
- D. No hay diferencia, son lo mismo

Retroalimentación:

En hosting compartido múltiples sitios comparten los mismos recursos del

servidor (CPU, RAM, disco). En VPS tienes recursos virtualizados dedicados, lo que ofrece mejor rendimiento y control, aunque requiere más conocimientos técnicos.

Pregunta 34

¿Cuáles son los tres tipos de entornos principales en un proyecto web?

- A. Frontend, backend y base de datos
- B. Solo desarrollo y producción
- C. Local, remoto y en la nube
- **D. Desarrollo, Testing/Staging y Producción**

 Retroalimentación:

Los tres entornos principales son: Desarrollo (tu PC), Testing/Staging (pruebas que imitan producción) y Producción (entorno real donde los usuarios acceden a la aplicación).

Pregunta 35

¿Qué ventaja principal tiene el hosting compartido para principiantes?

- A. Solo funciona con aplicaciones muy complejas
- B. Requiere conocimientos avanzados de Linux
- **C. No requiere configuración de servidores y tiene panel de control gráfico**
- D. Es el más caro pero más potente

 Retroalimentación:

El hosting compartido es ideal para principiantes porque no requiere configuración de servidores, incluye panel de control gráfico y permite centrarse en entender el proceso de despliegue.

Pregunta 36

¿Qué función cumple el DNS en una aplicación web?

- A. Almacena el código fuente de la aplicación
- B. Gestiona la base de datos del proyecto
- **C. Traduce nombres de dominio en direcciones IP reales**
- D. Encripta las comunicaciones HTTPS

 Retroalimentación:

El DNS (Domain Name System) actúa como la 'guía telefónica de Internet', traduciendo nombres legibles como 'devcampus.es' en direcciones IP reales donde está ubicado el servidor.

Pregunta 37

¿Por qué es importante que un sitio web tenga certificados SSL (HTTPS)?

- **A. Encripta la comunicación y es requerido por navegadores modernos**
- B. Solo lo necesitan los sitios de banca
- C. Solo es necesario para tiendas online
- D. Únicamente mejora el SEO

Retroalimentación:

Los certificados SSL encriptan la comunicación entre el navegador y el servidor, protegiendo datos sensibles. Además, los navegadores modernos marcan como 'no seguro' los sitios sin HTTPS.

Pregunta 38

¿Cuál es la diferencia principal entre desplegar una aplicación estática y una con backend?

- A. Solo la estática puede usar HTTPS
- B. La estática solo necesita subir archivos HTML/CSS/JS, la con backend requiere servidor de aplicaciones
- C. No hay diferencia, se despliegan igual
- D. La estática es más cara de mantener

Retroalimentación:

Una aplicación estática (HTML/CSS/JS) solo necesita un servidor web que sirva archivos. Una con backend requiere un servidor de aplicaciones (Node.js, PHP) que procese lógica y se conecte a bases de datos.

Pregunta 39

¿Qué es Docker y cuál es su principal función?

- A. Una herramienta que crea entornos reproducibles para ejecutar aplicaciones
- B. Una base de datos en la nube
- C. Un lenguaje de programación para aplicaciones web
- D. Un framework para desarrollo frontend

Retroalimentación:

Docker es una herramienta que crea contenedores con entornos controlados, permitiendo que las aplicaciones funcionen igual en diferentes sistemas (tu PC, servidor, etc.), solucionando el problema 'en mi ordenador funciona'.

Pregunta 40

¿Cuál es la diferencia principal entre un contenedor Docker y una máquina virtual?

- A. No hay diferencia, son lo mismo
- B. Un contenedor es más ligero, una máquina virtual es un ordenador completo dentro de otro
- C. Los contenedores solo funcionan en Linux
- D. Las máquinas virtuales son más rápidas

Retroalimentación:

Un contenedor Docker es una 'caja aislada' que incluye solo lo necesario para la aplicación, siendo más ligero y rápido. Una máquina virtual simula un ordenador completo con su propio sistema operativo.

Pregunta 41

¿Qué tipo de aplicaciones se pueden desplegar en Vercel?

- A. Únicamente aplicaciones PHP
- B. Solo aplicaciones de bases de datos
- C. Solo aplicaciones de escritorio
- D. Aplicaciones frontend estáticas y funciones serverless

 Retroalimentación:

Vercel es ideal para aplicaciones frontend estáticas (React, Vue, HTML/CSS/JS) y funciones serverless. No soporta servidores persistentes como Node.js con bases de datos tradicionales.

Pregunta 42

¿Por qué es importante documentar un proyecto de desarrollo web?

- A. Facilita el mantenimiento, comunicación y trabajo en equipo
- B. Únicamente para cumplir con regulaciones
- C. No es importante, el código se explica solo
- D. Solo es necesario para proyectos grandes

 Retroalimentación:

La documentación facilita el mantenimiento del proyecto, explica decisiones técnicas, acelera la incorporación de nuevos desarrolladores y ayuda a detectar incoherencias en el código.

Pregunta 43

¿Qué debe incluir como mínimo el archivo README.md de un proyecto?

- A. Únicamente el código fuente
- B. Solo información de contacto del autor
- C. Solo el nombre del proyecto
- D. Descripción del proyecto, instrucciones de instalación y configuración

 Retroalimentación:

Un buen README.md debe explicar qué es el proyecto, cómo instalarlo, cómo ejecutarlo, cómo configurarlo y cualquier información necesaria para que otros desarrolladores puedan trabajar con él.

Pregunta 44

¿Qué información debe contener un manual de instalación de un proyecto web?

- A. Requisitos del sistema, pasos de instalación, configuración y verificación
- B. Solo el comando 'npm install'
- C. Únicamente el enlace de descarga
- D. Solo información sobre el precio

 Retroalimentación:

Un manual de instalación completo debe incluir: requisitos previos (Node.js, bases de datos), pasos detallados de instalación, configuración necesaria (.env), y cómo verificar que todo funciona correctamente.

Pregunta 45

¿Cuál es la diferencia entre documentación técnica y documentación de usuario?

- A. Solo la documentación técnica es importante

- **B.** La técnica es para desarrolladores (arquitectura, APIs), la de usuario explica cómo usar la aplicación
- C. La técnica es más cara de hacer
- D. No hay diferencia, son lo mismo

 Retroalimentación:

La documentación técnica se dirige a desarrolladores y administradores (código, configuración, APIs). La de usuario explica a los usuarios finales cómo usar la aplicación (pantallas, flujos, acciones).

Pregunta 46

¿Cuál es la sintaxis correcta para insertar un nuevo registro en SQL?

- A. PUT INTO tabla (campo1, campo2) VALUES (valor1, valor2)
- B. ADD INTO tabla (campo1, campo2) VALUES (valor1, valor2)
- C. CREATE INTO tabla (campo1, campo2) VALUES (valor1, valor2)
- **D. INSERT INTO tabla (campo1, campo2) VALUES (valor1, valor2)**

 Retroalimentación:

INSERT INTO es la instrucción correcta para añadir nuevos registros en SQL. Se especifican los campos y sus valores correspondientes entre paréntesis.

Pregunta 47

¿Qué hace la instrucción UPDATE en SQL?

- A. Añade nuevas columnas
- B. Elimina registros de una tabla
- **C. Modifica registros existentes en una tabla**
- D. Crea una nueva tabla

 Retroalimentación:

UPDATE modifica registros existentes en una tabla SQL. Se usa junto con SET para especificar los nuevos valores y WHERE para indicar qué registros cambiar: UPDATE tabla SET campo = valor WHERE condición.

Pregunta 48

¿Qué hace el comando 'git status'?

- A. Sube cambios al servidor
- B. Borra archivos
- C. Crea un commit
- **D. Muestra el estado actual del repositorio y archivos modificados**

 Retroalimentación:

git status muestra qué archivos han cambiado, cuáles están en staging, en qué rama estás y el estado general del repositorio. Es fundamental para entender el estado actual.

Pregunta 49

¿Para qué se usa la instrucción DELETE en SQL?

- **A. Para modificar la estructura de una tabla**

- B. Para hacer copias de seguridad
- C. Para eliminar registros de una tabla
- D. Para crear nuevas tablas

 Retroalimentación:

DELETE elimina registros de una tabla SQL. Es importante usar siempre WHERE para especificar qué registros eliminar: DELETE FROM tabla WHERE condición. Sin WHERE eliminaría todos los registros.

Pregunta 50

¿Para qué se usa JOIN en SQL?

- A. Para combinar datos de múltiples tablas relacionadas
- B. Para eliminar duplicados
- C. Para ordenar resultados
- D. Para crear nuevas tablas

 Retroalimentación:

JOIN se usa para combinar datos de múltiples tablas que están relacionadas mediante claves. Por ejemplo: SELECT * FROM usuarios JOIN pedidos ON usuarios.id = pedidos.usuario_id obtiene datos de ambas tablas.

Ver Resultados Finales



¡Excelente!

Puntuación final: 50/50 (100%)

Preguntas respondidas: 50/50

Correctas: 50 | Incorrectas: 0